

Jaypee Institute of Information Technology, Noida

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION
TECHNOLOGY**

Project Title: Console Browser Engine Simulation



Submitted By:

Enrolment No.: 2501030359, Name: ANUSHKA LAL

Enrolment No.: 2501030348, Name: MAHEE SINGH RATHORE

Enrolment No.: 2501030347, Name: SUYASH GUPTA

Course Name: Software Development Lab-II

Course Code: 24B15CS121

Program: B. Tech. CSE/ECS/ECE

First Year, 2nd Semester

Chapter 1: Introduction

1.1 Background

The World Wide Web and internet browsers are among the most transformative technologies of the modern era. A web browser is a complex software application responsible for fetching web resources, parsing HTML/CSS, and rendering visual output for end users. Traditional browser development involves intricate multi-layered architecture including a network layer, caching system, rendering engine, DOM parser, and user interface management.

This project simulates a simplified console-based browser engine in C++ to demonstrate and apply core Object-Oriented Programming (OOP) principles. While not a functional internet browser, it replicates the essential architectural patterns found in real browsers such as Chromium and Mozilla Firefox, making it a strong pedagogical tool for understanding system design.

1.2 Problem Statement

Modern web browser architecture is rarely taught through hands-on implementation at the undergraduate level. Students learning OOP concepts often lack exposure to real-world system design that leverages inheritance, polymorphism, templates, and design patterns in a cohesive, meaningful project.

This project addresses the gap by implementing a layered browser simulation where each layer has a clearly defined responsibility, demonstrating how OOP enables modular, maintainable, and extensible software design. The problem this project solves is: How can OOP concepts be used to design a multi-layered, functional simulation of a web browser in C++?

1.3 Social & Environmental Relevance

This project directly aligns with SDG 9: Industry, Innovation and Infrastructure. SDG 9 promotes building resilient infrastructure, fostering inclusive and sustainable industrialization, and encouraging innovation. The browser engine simulation embodies all three sub-targets:

- **Resilient Infrastructure:** The layered architecture (network, cache, rendering) mirrors real-world browser infrastructure design. Understanding this from first principles enables future engineers to build robust digital infrastructure.

- **Innovation:** The simulation introduces students to advanced C++ features (templates, polymorphism, factory methods) that are directly relevant in the software industry, nurturing an innovative mindset for system design.
- **Industrialization:** A strong pipeline of trained software engineers capable of building reliable systems is essential for the digital industrialization of developing economies, including India.

Additionally, the project indirectly supports SDG 4 (Quality Education) by serving as an applied learning tool that transforms abstract OOP theory into hands-on system design experience.

Primary SDG: SDG 9 – Industry, Innovation and Infrastructure

Secondary SDGs: SDG 4 – Quality Education

1.4 Scope

The project covers the following:

- A fully functional console-based browser simulation with multi-tab support
- User authentication system (signup/login using file I/O)
- Network layer with a page database and LRU-style cache
- Rendering layer with polymorphic web page classes (News, Shop, Sports, Quotes, Calendar, Game)
- DOM parser and HTML renderer for page inspection
- History navigation using a doubly linked list
- Tab management using a circular doubly linked list
- Themed console output using ANSI escape codes
- Tic-tac-toe game embedded as an interactive page

Limitations: The browser cannot access the real internet; pages are pre-defined. It is a Windows-targeted application (uses `windows.h` and `system("cls")`).

Chapter 2: System Design & Methodology

2.1 Proposed Methodology

The browser engine follows a layered, pipeline-based architecture. Each layer is independently encapsulated and communicates through well-defined interfaces, closely mirroring real-world browser design.

System Architecture Overview:

BROWSER ENGINE - LAYERED ARCHITECTURE

```
[main.cpp] → Browser (browser_engine.h)
|
+---> UserManager (Auth / File I/O)
+---> Tab<T> (Circular DLL, HistoryManager)
+---> RequestPipeline (network_layer.h)
|           +---> Cache (map-based)
|           +---> PageDatabase (map-based)
+---> WebPage* createPage() [Factory]
|   └─ NewsPage / ShopPage / SportsPage / ...
+---> DOMParser + HTMLRenderer (rendering_layer.h)
```

Request Processing Flow:

1. User enters a URL via the console.
2. Browser calls RequestPipeline::process(url).
3. Pipeline checks Cache; on HIT returns cached Page, on MISS fetches from PageDatabase.
4. Page content is passed to DOMParser to build a DOMNode tree.
5. Factory method createPage() instantiates the correct WebPage subclass.
6. WebPage::render() outputs the UI using card() and topBar() helpers.

2.2 Tools and Technique Selection

Language & Compiler: C++17 using g++ on Windows. C++ was chosen for its native support of OOP, templates, manual memory management, and ANSI console control.

Data Structures Used:

- `std::map`: Used in PageDatabase and Cache for $O(\log n)$ URL lookup.
- Doubly Linked List (custom): Used in HistoryManager for back/forward navigation.
- Circular Doubly Linked List (custom): Used in the Browser for multi-tab management.

OOP Concepts Applied:

OOP Concept	Where Applied	Purpose
Abstraction	WebPage (abstract class)	Defines render() interface; hides page-type specifics
Inheritance	NewsPage, ShopPage, SportsPage etc. extend WebPage	Enables code reuse and specialisation
Polymorphism	WebPage* pointer calls correct render()	Runtime dispatch to the right page type
Encapsulation	Cache, PageDatabase, HistoryManager classes	Internal data hidden
Templates	Tab<T> class template	Type-generic tab; demonstrating parameterised types
Factory Method	Browser::createPage(url)	Decouples object creation from usage
Composition	Browser holds Cache, Pipeline, Renderer, Parser	Builds complex system from simpler objects

Chapter 3: Implementation & Testing

3.1 Experimental Setup

Component	Details
Operating System	Windows 10/11 (64-bit)
Programming Language	C++17
Compiler	g++ (MinGW-w64 / MSVC)
IDE	Visual Studio Code / Code::Blocks
Libraries Used	<iostream>, <map>, <vector>, <ctime>, <fstream>, <windows.h>
Input Method	Console (stdin/stdout)
Persistence	users.bin (plaintext file for credentials), page.txt (print output)
Memory Management	Manual (new/delete) for linked list nodes

3.2 Results (Test Cases / Expected Output)

The following test cases validate core browser functionality:

T C #	Test Case	Input	Expected Output	Status
T C 1	Valid URL (first visit)	www.news.com	[Network Fetch] + NewsPage rendered	Pass
T C 2	Same URL (cached)	www.news.com (again)	[Cache HIT] + page rendered	Pass
T C 3	Unknown but valid URL	www.abc.com	[Generic Page fallback] rendered	Pass

T C 4	Invalid URL format	google.com	[Error] Invalid URL format	Pass
T C 5	Back navigation	Back after visiting 3 URLs	Previous URL loads correctly	Pass
T C 6	Forward navigation	Forward after going back	Next URL loads correctly	Pass
T C 7	User signup	Valid email + 8-char password	Saved to users.bin, home opened	Pass
T C 8	Login with wrong password	Wrong credentials	"Login failed" shown	Pass
T C 9	New tab + switch	Open new tab, switch to it	New tab at www.home.com active	Pass
T C 10	Close tab	Close active tab (2 open)	Tab removed, next tab active	Pass

All test cases passed successfully. The system demonstrated correct cache behaviour, polymorphic rendering, history management, and error handling.

Chapter 4: Conclusion & Future Scope

4.1 Summary of Work and Achievements

This project successfully implemented a console-based browser engine simulation in C++ that demonstrates how Object-Oriented Programming concepts can be combined to model a real-world, multi-layered software system.

OOP contributions to solving the problem:

- **Abstraction:** The abstract `WebPage` class defines a contract (`render()`) that every page type must fulfill, allowing the browser to render any page without knowing its concrete type.
- **Polymorphism:** At runtime, calling `page->render(url)` dispatches to the correct subclass (`NewsPage`, `ShopPage`, etc.), enabling new page types to be added with zero changes to the browser core.
- **Templates:** The `Tab<T>` template generalises tab management, demonstrating type-generic programming.
- **Encapsulation:** Each subsystem (`Cache`, `PageDatabase`, `HistoryManager`, `UserManager`) owns its data and exposes only the necessary interface, preventing unintended state corruption.
- **Factory Method Pattern:** `createPage()` centralises object creation, making the rendering subsystem easy to extend.
- **Composition:** The `Browser` class is assembled from several collaborating objects, demonstrating how large systems are built from smaller, testable units.

The project achieves a working simulation of core browser behaviour: URL navigation, caching, history, multi-tab management, authentication, and polymorphic page rendering.

4.2 Impact on Society, Environmental Sustainability, Ethical Issues and Compliance

Social Impact: By providing students with a hands-on, applied OOP project that mirrors real-world browser architecture, this work contributes directly to SDG 9 (Industry, Innovation and Infrastructure) and SDG 4 (Quality Education). Engineers trained in layered system design are better equipped to build the digital infrastructure that underpins modern economies.

Environmental Sustainability: The console-based approach requires minimal computational resources compared to GUI applications. The simulation runs efficiently without network calls, reducing unnecessary data transmission and energy consumption in a lab/educational setting.

Ethical Issues: The authentication system stores passwords in plaintext (users.bin), which is ethically and practically insecure. Future versions must use hashing (e.g., bcrypt or SHA-256). Users should be informed of data storage practices transparently.

Compliance: The project is purely academic and does not collect or transmit user data over any network, making it compliant with data protection principles in its current scope.

4.3 Limitations of the Work and Future Scope

Current Limitations:

- Pages are statically defined; the browser cannot access real internet resources.
- Passwords stored in plaintext (users.bin) – a significant security risk.
- Windows-only (uses windows.h, system("cls"), Sleep()); not cross-platform.
- No support for CSS, JavaScript, images, or multimedia content.
- The Tic-Tac-Toe game does not implement a win-detection algorithm.
- No memory leak protection; manual new/delete may cause leaks on abnormal exit.

Future Scope:

- Cross-Platform Support: Replace Windows-specific calls with platform-agnostic alternatives (e.g., POSIX sleep, ncurses for terminal UI).
- Secure Authentication: Implement password hashing using a library such as OpenSSL.
- Real HTTP Requests: Integrate libcurl to fetch actual web content.
- GUI Mode: Replace console output with an SDL2 or Qt-based graphical interface.
- Extended Game Library: Add a full-featured game page with win detection and AI opponent.
- Tabbed History Isolation: Each tab currently maintains independent history; adding a shared global history panel would enhance UX.

References

- [1] B. Stroustrup, The C++ Programming Language, 4th ed. Addison-Wesley, 2013.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [3] United Nations, "Sustainable Development Goal 9: Industry, Innovation and Infrastructure," UN SDGs, 2015. [Online]. Available: <https://sdgs.un.org/goals/goal9>
- [4] United Nations, "Sustainable Development Goal 4: Quality Education," UN SDGs, 2015. [Online]. Available: <https://sdgs.un.org/goals/goal4>
- [5] Google Chromium Project, "Multi-process Architecture," Chromium Documentation. [Online]. Available: <https://www.chromium.org/developers/design-documents/multi-process-architecture>
- [6] Mozilla Developer Network, "How Browsers Work," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Performance/How_browsers_work

Appendix

Team Roles and Contributions

Member	Enrolment No.	Contribution
ANUSHKA LAL	2501030359	Network Layer, Cache, Pipeline, DOM Parser
MAHEE SINGH RATHORE	2501030348	Browser Engine Features, Tab/History management, Auth, Theme system
SUYASH GUPTA	2501030347	Rendering Layer, WebPage classes, PageDatabase

```
=====
CONSOLE BROWSER
=====
1.Sign Up
2.Login
3.Continue as Guest
4.Inspect
0.Exit
```

```
[ home ] |
=====
1.New URL
2.View Shortcuts
3.Switch Tabs
4.Settings
0.Logout
|
```

```
+-----+
| URL: www.shop.com |
+-----+
+-----+
| Electronics |
+-----+
| Phones |
| Laptops |
| Tablets |
+-----+
+-----+
| Fashion |
+-----+
| Minimal wear trending |
+-----+
+-----+
| Books |
+-----+
| Fiction |
| Self-help |
+-----+

WELCOME suyash,

[ shop ] |
=====
1.Back 2.Forward 3.New Tab 4.Switch Tab 5.Inspect 6.Close 7.Print 0.Home
|
```

```
SETTINGS
-----
1.History 2.Theme 0.Back
1

--- SESSION HISTORY ---
-> www.shop.com
-----

Press x to exit...
|
```

```
WELCOME suyash,

  shop  | [ game ] |
=====
1.Back  2.Forward  3.New Tab  4.Switch Tab  5.Inspect  6.Close  7.Print  0.Home
|
```

```
=====
[Cache HIT]
html
  body
    Game Page
=====
WELCOME suyash,

  shop  | [ game ] |
=====
1.Back  2.Forward  3.New Tab  4.Switch Tab  5.Inspect  6.Close  7.Print  0.Home
|
```

```
[ shop ] | game |
=====
1.Back  2.Forward  3.New Tab  4.Switch Tab  5.Inspect  6.Close  7.Print  0.Home
4
1.Prev  2.Next
2|
```

```
[Cache HIT]
Printed
WELCOME suyash,

  shop  | [ game ] |
=====
1.Back  2.Forward  3.New Tab  4.Switch Tab  5.Inspect  6.Close  7.Print  0.Home
```

```
SETTINGS
-----
1.History 2.Theme 0.Back
2
1.Default
2.Dark Console
3.Hacker Green
4.Crimson Red
5. Midnight Blue
```

```
WELCOME suyash,

  shop  | [ home ] |
=====
1.New URL
2.View Shortcuts
3.Switch Tabs
4.Settings
0.Logout
```

```
+-----+
| URL: www.word.com |
+-----+
| Generic Page |
+-----+
| This is a dynamically generated page. |
| No customised template currently exists for this site ... |
| Since valid url, hence, Rendered using base class Gene... |
+-----+

WELCOME suyash,

  shop  | [ word ] |
=====
1.Back 2.Forward 3.New Tab 4.Switch Tab 5.Inspect 6.Close 7.Print 0.Home
```